

Gerando matrizes ortonormais aleatórias com o pacote rstiefel

Vítor Vieira

```
# Install from CRAN (if available for your R version)
#install.packages("devtools")
#devtools::install_github("pdhoff/rstiefel")

library(rstiefel)

# Suppose we want an m x k orthonormal matrix U with m = 10 rows, k = 5 columns
m <- 10
k <- 5
M <- matrix(rnorm(m * k), m, k) # just any numeric matrix as a template

U <- rmf.matrix(M)

# Check orthonormality: U^T U should be ~ Identity_k
t(U) %*% U
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,]  1.000000e+00  1.697861e-16  2.263814e-16 -7.164408e-16  2.486076e-16
## [2,]  1.697861e-16  1.000000e+00  4.718448e-16 -4.163336e-16 -4.163336e-17
## [3,]  2.263814e-16  4.718448e-16  1.000000e+00  1.110223e-16 -5.551115e-16
## [4,] -7.164408e-16 -4.163336e-16  1.110223e-16  1.000000e+00  1.110223e-16
## [5,]  2.486076e-16 -4.163336e-17 -5.551115e-16  1.110223e-16  1.000000e+00
```

```
#####
## Bayesian SVD Gibbs Sampler (rank K fixo)
## usando rstiefel para amostrar/atualizar U e V na variedade
## de Stiefel (colunas ortonormais)
##
## Modelo:
##   Y = U D V^T + E
##   E_ij ~ Normal(0, 1/phi)
##
## Priors (idéia Hoff 2007):
##   U, V ~ uniformes na Stiefel (colunas ortonormais)
##   d_j | mu, psi ~ Normal(mu, 1/psi)
##   mu ~ Normal(mu0, v0_sq)
##   psi ~ Gamma(shape=(eta0/2), rate=(eta0*tau0_sq/2))
##   phi ~ Gamma(shape=(nu0/2), rate=(nu0*sigma0_sq/2))
##
## Passos do Gibbs:
##   1. U | resto ~ Bingham-von Mises-Fisher -> rbmf.matrix.gibbs
##   2. V | resto ~ Bingham-von Mises-Fisher -> rbmf.matrix.gibbs
##   3. d_j | resto é Normal(...)
##   4. phi | resto é Gama(...)
```

```

## 5. mu | d,psi é Normal(...)
## 6. psi | d,mu é Gama(...)
##
## ATENÇÃO:
## - A construção de (A,B,C) para rbmf.matrix.gibbs é consistente com a forma
##   quadrática+linear típica da condicional de U e V, mas é uma aproximação
##   prática. A ideia é manter U e V ortonormais e "puxando" na direção que
##   melhora o ajuste  $Y \sim UDV^T$ .
#####

library(rstiefel)

#####
## Função auxiliar: média posterior de  $M = U \text{diag}(d) V^T$ 
## Corrigida para não dar erro de dimensão
#####
posterior_mean_M <- function(samples) {
  U_arr <- samples$U      # m x K x n_keep
  V_arr <- samples$V      # n x K x n_keep
  d_mat <- samples$d      # n_keep x K

  m <- dim(U_arr)[1]
  K <- dim(U_arr)[2]
  n_keep <- dim(U_arr)[3]
  n <- dim(V_arr)[1]

  M_bar <- matrix(0, m, n)

  for (s in 1:n_keep) {
    # drop=TRUE transforma cada fatia em matriz 2D (não array 3D)
    U_s <- U_arr[,s, drop=TRUE] # m x K
    V_s <- V_arr[,s, drop=TRUE] # n x K
    d_s <- d_mat[s, ]           # vetor de tamanho K

    # Reconstrói  $M_s = U_s \text{diag}(d_s) V_s^T$ 
    M_s <- U_s %*% diag(d_s, nrow=K) %*% t(V_s) # m x n

    M_bar <- M_bar + M_s
  }

  M_bar / n_keep
}

#####
## Gibbs Sampler principal
#####
bayes_svd_gibbs_rstiefel <- function(Y, K,
                                     n_iter = 2000,
                                     burn_in = 1000,
                                     # Hiperparâmetros
                                     nu0 = 2,
                                     sigma0_sq = 1,
                                     mu0 = 0,

```

```

v0_sq      = 1,
eta0       = 2,
tau0_sq    = 1,
verbose    = FALSE) {

m <- nrow(Y)
n <- ncol(Y)

# --- Inicialização de U e V usando rstiefel::rmf.matrix ---
# rmf.matrix() gera matriz ortonormal (colunas ortonormais) no formato pedido
U <- rmf.matrix(matrix(rnorm(m * K), m, K)) # m x K
V <- rmf.matrix(matrix(rnorm(n * K), n, K)) # n x K

# Valores iniciais de d, phi, psi, mu
d <- rnorm(K)
phi <- 1 # precisão do ruído
psi <- 1 # precisão das d_j em torno de mu
mu <- 0 # média das d_j

# Armazenamento pós burn-in
keep_iter <- (burn_in+1):n_iter
n_keep <- length(keep_iter)

U_store <- array(NA_real_, dim = c(m, K, n_keep))
V_store <- array(NA_real_, dim = c(n, K, n_keep))
d_store <- matrix(NA_real_, nrow = n_keep, ncol = K)
phi_store <- numeric(n_keep)
mu_store <- numeric(n_keep)
psi_store <- numeric(n_keep)

keep_idx <- 1

for (iter in 1:n_iter) {

#####
## 1. Atualizar U | (Y, V, d, phi)
##
## Precisamos passar A (m x m), B (K x K diag), C (m x K)
## para rbmf.matrix.gibbs. Isso faz uma varredura Gibbs
## em U respeitando  $U^T U = I_K$ .
#####
A_U <- Y %*% t(Y) # m x m
B_U <- diag(sort(abs(d), decreasing=TRUE), K) # K x K diag
C_U <- Y %*% (V %*% diag(d, nrow=K)) # m x K

U <- rbmf.matrix.gibbs(
  A = A_U,
  B = B_U,
  C = C_U,
  X = U
)

#####

```

```

## 2. Atualizar V | (Y, U, d, phi)
##
## Agora V é n x K, então A_V é n x n, etc.
#####
A_V <- t(Y) %*% Y # n x n
B_V <- diag(sort(abs(d), decreasing=TRUE), K) # K x K diag
C_V <- t(Y) %*% (U %*% diag(d, nrow=K)) # n x K

V <- rbmf.matrix.gibbs(
  A = A_V,
  B = B_V,
  C = C_V,
  X = V
)

#####
## 3. Atualizar cada d_j | resto
##
## Condicional Normal:
## var_j = 1 / (phi + psi)
## mean_j = (phi * signal_j + psi * mu) / (phi + psi)
##
## onde signal_j = U[,j]^T E_{-j} V[,j],
## e E_{-j} é Y menos todas as outras componentes k != j.
#####
for (j in 1:K) {

  U_minusj <- U[, -j, drop=FALSE]
  V_minusj <- V[, -j, drop=FALSE]
  d_minusj <- d[-j]

  if (length(d_minusj) == 0) {
    E_minusj <- Y
  } else {
    E_minusj <- Y -
      U_minusj %*% diag(d_minusj, nrow=length(d_minusj)) %*%
      t(V_minusj)
  }

  signal_j <- drop(
    t(U[, j, drop=FALSE]) %*% E_minusj %*% V[, j, drop=FALSE]
  )

  post_var_j <- 1 / (phi + psi)
  post_mean_j <- (signal_j * phi + mu * psi) / (phi + psi)

  d[j] <- rnorm(1, mean = post_mean_j, sd = sqrt(post_var_j))
}

#####
## 4. Atualizar phi | resto
##
## R = Y - U diag(d) V^T

```

```

## phi ~ Gama( (nu0+m*n)/2 , (nu0*sigma0_sq + ||R||^2)/2 )
#####
R <- Y - U %*% diag(d, nrow=K) %*% t(V)
ss <- sum(R * R)

shape_phi <- (nu0 + m * n) / 2
rate_phi <- (nu0 * sigma0_sq + ss) / 2
phi <- rgamma(1, shape=shape_phi, rate=rate_phi)

#####
## 5. Atualizar mu | d, psi
##
## mu ~ Normal(post_mean_mu, post_var_mu)
## post_var_mu = 1 / (psi*K + 1/v0_sq)
## post_mean_mu = (psi*sum(d) + mu0/v0_sq)*post_var_mu
#####
post_var_mu <- 1 / (psi * K + 1/v0_sq)
post_mean_mu <- (psi * sum(d) + mu0 / v0_sq) * post_var_mu
mu <- rnorm(1, mean = post_mean_mu, sd = sqrt(post_var_mu))

#####
## 6. Atualizar psi | d, mu
##
## psi ~ Gama( (eta0+K)/2 , (eta0*tau0_sq + sum((d-mu)^2))/2 )
#####
ss_d <- sum((d - mu)^2)
shape_psi <- (eta0 + K) / 2
rate_psi <- (eta0 * tau0_sq + ss_d) / 2
psi <- rgamma(1, shape=shape_psi, rate=rate_psi)

if (verbose && (iter % 100 == 0)) {
  cat(sprintf("iter %d | phi=%.3f mu=%.3f psi=%.3f d=%s\n",
              iter, phi, mu, psi,
              paste(round(d,3), collapse=",")))
}

#####
## Salvar amostras pós burn-in
#####
if (iter %in% keep_iter) {
  U_store[,keep_idx] <- U
  V_store[,keep_idx] <- V
  d_store[keep_idx,] <- d
  phi_store[keep_idx] <- phi
  mu_store[keep_idx] <- mu
  psi_store[keep_idx] <- psi
  keep_idx <- keep_idx + 1
}
}

list(
  U = U_store,    # m x K x n_keep amostras
  V = V_store,    # n x K x n_keep amostras

```

```

    d = d_store, # n_keep x K
    phi = phi_store, # n_keep
    mu = mu_store, # n_keep
    psi = psi_store, # n_keep
    Y = Y,
    K = K,
    burn_in = burn_in
  )
}

#####
## EXEMPLO DE USO (DESCOMENTADO)
#####

set.seed(123)

m <- 10
n <- 8
K <- 3

# Gerar sinal verdadeiro low-rank
true_U <- rmf.matrix(matrix(rnorm(m * K), m, K)) # m x K ortonormal
true_V <- rmf.matrix(matrix(rnorm(n * K), n, K)) # n x K ortonormal
true_d <- c(3, 2, 0.5) # "valores singulares" verdadeiros

M_true <- true_U %*% diag(true_d, nrow=K) %*% t(true_V) # m x n

phi_true <- 5 # precisão do ruído => variância = 1/phi_true
Y <- M_true + matrix(rnorm(m * n, sd = sqrt(1/phi_true)), m, n)

# Rodar o Gibbs
fit <- bayes_svd_gibbs_rstiefel(
  Y, K,
  n_iter = 3000,
  burn_in = 1500,
  nu0 = 2,
  sigma0_sq = 1,
  mu0 = 0,
  v0_sq = 1,
  eta0 = 2,
  tau0_sq = 1,
  verbose = TRUE
)

## iter 100 | phi=4.102 mu=1.295 psi=0.077 d=3.874,3.896,0.546
## iter 200 | phi=4.208 mu=2.323 psi=0.085 d=3.211,3.138,-0.041
## iter 300 | phi=7.623 mu=0.060 psi=0.103 d=2.844,3.534,0.437
## iter 400 | phi=3.845 mu=-0.674 psi=0.391 d=2.399,2.088,0.745
## iter 500 | phi=4.557 mu=1.611 psi=2.283 d=2.356,2.073,0.608
## iter 600 | phi=3.875 mu=0.406 psi=0.757 d=1.76,2.451,0.348
## iter 700 | phi=4.975 mu=1.610 psi=0.988 d=3.321,2.278,0.792
## iter 800 | phi=6.029 mu=-0.239 psi=0.110 d=3.156,2.849,-0.189
## iter 900 | phi=5.380 mu=0.644 psi=0.464 d=2.886,3.167,0.241

```

```
## iter 1000 | phi=3.411 mu=1.535 psi=0.576 d=2.772,2.23,-0.089
## iter 1100 | phi=6.520 mu=0.058 psi=0.716 d=2.488,3.069,-0.077
## iter 1200 | phi=5.142 mu=-0.829 psi=0.021 d=2.822,2.157,0.17
## iter 1300 | phi=6.648 mu=1.831 psi=0.400 d=2.76,3.273,0.845
## iter 1400 | phi=6.106 mu=1.776 psi=0.996 d=3.82,2.999,0.065
## iter 1500 | phi=4.342 mu=1.017 psi=0.335 d=2.844,2.203,-0.892
## iter 1600 | phi=4.154 mu=1.008 psi=1.236 d=2.688,1.926,0.172
## iter 1700 | phi=4.599 mu=1.503 psi=0.264 d=3.043,3.327,0.314
## iter 1800 | phi=7.374 mu=2.228 psi=0.610 d=2.925,3.407,0.694
## iter 1900 | phi=3.583 mu=0.223 psi=0.246 d=3.053,3.209,-0.53
## iter 2000 | phi=7.833 mu=0.162 psi=0.214 d=3.277,3.184,0.598
## iter 2100 | phi=5.114 mu=1.223 psi=1.122 d=2.514,2.512,1.014
## iter 2200 | phi=6.468 mu=0.247 psi=0.171 d=3.93,3.505,-0.392
## iter 2300 | phi=6.148 mu=1.912 psi=0.443 d=2.98,2.955,1.216
## iter 2400 | phi=4.209 mu=1.250 psi=0.313 d=2.734,3.079,-0.633
## iter 2500 | phi=3.883 mu=0.216 psi=0.239 d=2.702,2.398,0.777
## iter 2600 | phi=5.692 mu=0.865 psi=0.230 d=2.983,2.496,0.873
## iter 2700 | phi=4.971 mu=2.186 psi=0.961 d=2.572,2.407,-0.175
## iter 2800 | phi=5.785 mu=1.377 psi=0.369 d=3.694,3.096,-0.989
## iter 2900 | phi=4.699 mu=1.691 psi=0.456 d=3.55,3.049,0.083
## iter 3000 | phi=4.152 mu=0.228 psi=0.335 d=2.995,2.97,0.469
```

```
# Calcular a média posterior de  $M = U \text{diag}(d) V^T$ 
M_hat <- posterior_mean_M(fit)

# Comparar reconstrução com a matriz verdadeira  $M_{\text{true}}$ 
rmse <- sqrt(mean((M_hat - M_true)^2))
cat("RMSE da reconstrução posterior vs verdade:", rmse, "\n")
```

```
## RMSE da reconstrução posterior vs verdade: 0.2660945
```

```
# Conferir ortonormalidade média da última amostra salva
last_idx <- dim(fit$U)[3]
U_last <- fit$U[,last_idx, drop=TRUE]
V_last <- fit$V[,last_idx, drop=TRUE]

cat("t(U_last) %*% U_last ~ I ?\n")
```

```
## t(U_last) %*% U_last ~ I ?
```

```
print(t(U_last) %*% U_last)
```

```
##           [,1]      [,2]      [,3]
## [1,] 1.000000e+00 -2.596664e-17 8.294147e-18
## [2,] -2.596664e-17 1.000000e+00 -6.938894e-18
## [3,] 8.294147e-18 -6.938894e-18 1.000000e+00
```

```
cat("t(V_last) %*% V_last ~ I ?\n")
```

```
## t(V_last) %*% V_last ~ I ?
```

```
print(t(V_last) %*% V_last)
```

```
##           [,1]           [,2]           [,3]
## [1,]  1.000000e+00  1.147052e-16 -8.803722e-17
## [2,]  1.147052e-16  1.000000e+00  8.673617e-17
## [3,] -8.803722e-17  8.673617e-17  1.000000e+00
```